

UE5 C++ MASTERY ÇALIŞMA MERKEZİ · SUJET

03

DUNGEON ESCAPE

Composition mimarisiyle çalışan, delegate'lerle haberleşen, envanter ve kilit sistemine sahip birinci şahıs bir bulmaca odası kurmak.

SÜRE

6 saat 50 dakika

DERS SAYISI

50 ders

ALİŞTIRMA

8 zorunlu + 1 bonus

XP

150 (+35 bonus)

BÖLÜM 0 — GENEL BİLGİLER VE KURALLAR

0.1 Önsöz

Bu bölüm, kursun mimari dönüm noktasıdır. Artık tek bir Actor içinde her şeyi yazmak yerine, küçük, tekrar kullanılabilir parçalar (component) inşa edip bunları pointer'lar ve delegate'lerle birbirine bağlayacaksın. Bu, gerçek oyun stüdyolarında kullanılan mimarinin ta kendisidir — burada öğrendiğin desenler bu kurstan çok daha uzun ömürlü.

0.2 Amaç

Actor Component, pointer, delegate, Enhanced Input ve trace kavramlarını kullanarak; basılınca bir kapıyı tetikleyen bir basınç plakası, ışınla etkileşime giren bir toplama sistemi ve TArray tabanlı bir envanter ile çalışan bütün bir kaçış odası kurmak.

0.3 Genel Yönlendirmeler

- Her component'i yazmadan önce “kim kime ne söylüyor” diyagramını kâğıda çiz.
- Pointer kullandığın her yerde “bu pointer null olabilir mi?” sorusunu kendine sor.
- Delegate'leri test ederken önce bilerek bağlamayı unut, olayın tetiklenmediğini gözlemle.
- Composition'ı tercih et: yeni bir davranış için önce “bir component olabilir mi?” diye düşün.

0.4 Değerlendirme Kriterleri (Kendi Kendine Savunma)

Bir 42 öğrencisinin projesini savunması gibi, aşağıdaki sorulara videoya veya nota bakmadan, kendi cümlelerinle cevap verebilmelisin:

- Inheritance ile composition arasındaki fark nedir ve bu bölümde composition'ı nerede kullandın?
- Bir delegate'in Broadcast edilmesi ile bir fonksiyonun doğrudan çağırılması arasındaki fark nedir?
- Pass by value ile pass by reference arasındaki farkı, FHitResult örneği üzerinden anlatabilir misin?
- Cast<> başarısız olduğunda ne döner ve bunu kontrol etmezsen ne olur?

0.5 Zorunlu ve Yasak

✓ ZORUNLU

- Hareket/tetikleme mantığını ayrı Actor Component'lerde tutmak (composition)
- Component'ler arası iletişimde delegate kullanmak
- Her Cast<> sonucunu null kontrolünden geçirmek

✗ YASAK

- Trigger mantığını doğrudan kapı Actor'ünün içine gömmek (composition yerine copy-paste)
- Bir delegate yerine component'lerin birbirinin private üyelerine doğrudan erişmesi
- Cast<> sonucunu kontrolsüz kullanmak (null pointer erişimi riski)

0.6 XP ve Rank Sistemi

Bu bölümün her alıştırması bir XP değeri taşır. Zorunlu kısmı eksiksiz bitirince **150 XP** kazanırsın; bonus dahil üst sınır **185 XP**'dir (bonus, ancak zorunlu %100 tamamlandıktan sonra

açılır). Kazandığın XP'yi Notion'daki “XP Takip Sistemi” veritabanında işaretleyerek genel kurs rankını görebilirsin — bu bölümü bonusuyla bitirince **Sistem Programcısı** rankına yaklaşırsın.

BÖLÜM I — ZORUNLU KISIM

Alıştırmalar birbiri üzerine inşa olacak şekilde sıralanmıştır — bir öncekini bitirmeden sonrakine geçme. Her alıştırma, ilgili Notion sayfasındaki “Konu Anlatımı” bölümünün karşılık gelen kısmına dayanır.

EXERCISE 00 · 15 XP

Hareket Ettirici Component

İlgili dersler: Actor Components · Pointers 101 · Pointer Practice · GetOwner

Amaç — Herhangi bir Actor'e eklenebilen, sahibini hedefe doğru hareket ettiren bir Actor Component yazmak.

► Yapılacaklar

- Yeni bir UActorComponent alt sınıfı oluştur.
- GetOwner() ile sahibi olan Actor'e bir pointer al.
- TickComponent içinde sahibi hedef konuma doğru hareket ettir (Obstacle Assault'taki mantığı taşı).

⚠ Kısıtlar

- GetOwner() sonucunu kontrolsüz kullanma; teorik olarak nullptr dönebilir.

✓ Beklenen Sonuç

- Aynı component, herhangi bir Actor'e eklendiğinde onu hareket ettirebiliyor.

EXERCISE 01 · 20 XP

Trigger Component

İlgili dersler: Box Component · The Constructor · FindComponentByClass

Amaç — Bir Box Component kullanarak çarpışmayı algılayan ve aynı Actor üzerindeki Mover Component'e erişen bir Trigger Component yazmak.

► Yapılacaklar

- Constructor içinde CreateDefaultSubobject ile bir Box Component oluştur.
- FindComponentByClass ile aynı Actor üzerindeki Mover Component'i bul.
- Box'ın OnComponentBeginOverlap/EndOverlap olaylarına bağlan.

⚠ Kısıtlar

- FindComponentByClass sonucunu her TickComponent'te tekrar tekrar çağırma — BeginPlay'de bir kez bul, sakla.

✓ Beklenen Sonuç

- Trigger tetiklendiğinde Mover Component'in hareket bayrağı doğru değişiyor.

EXERCISE 02 · 20 XP

Delegate Bağlantısı

İlgili ders: Using Delegates

Amaç — Trigger Component'in, kendisini tetikleyen ne olduğunu bilmeden bir olay yayınlamasını sağlamak.

► Yapılacaklar

- DECLARE_DYNAMIC_MULTICAST_DELEGATE ile bir delegate tipi tanımla.
- Trigger Component'te bu tipte bir UPROPERTY delegate instance'ı oluştur.
- Tetiklendiğinde Broadcast() çağır; dinleyen taraf BeginPlay'de AddDynamic ile bağlansın.

⚠ Kısıtlar

- Dinleyen tarafın Trigger Component'in iç detaylarını bilmesini gerektirecek bir tasarım kurma.

✓ Beklenen Sonuç

- Kapı Actor'ü, Trigger Component'in ne olduğunu bilmeden sadece delegate'i dinleyerek tepki veriyor.

EXERCISE 03 · 15 XP

Basınç Plakası

İlgili dersler: Keeping Count · Public & Private · BP_PressurePlate

Amaç — Üzerinde birden fazla Actor durabilen, sayaç ile takip edilen bir basınç plakası kurmak.

► Yapılacaklar

- Plaka üzerindeki Actor sayısını private bir int ile tut.
- Overlap Begin'de artır, End'de azalt.
- Sayaç sıfıra düşmeden plakayı deaktif etme; sadece sıfırken tetikle.

⚠ Kısıtlar

- Sayaç değişkenini public yapıp dışarıdan serbestçe değiştirilebilir bırakma — encapsulation'ı koru.

✓ Beklenen Sonuç

- İki Actor plaka üzerindeyken biri ayrılrsa bile plaka aktif kalıyor.

EXERCISE 04 · 15 XP

Enhanced Input Kurulumu

İlgili dersler: Enhanced Input System · Input System in C++

Amaç — Oyuncunun bir “Etkileş” eylemini E tuşuna bağlamak için Enhanced Input asset'lerini oluşturup C++'a bağlamak.

► Yapılacaklar

- Bir Input Action ve bir Input Mapping Context asset'i oluştur.
- Mapping Context'i Player Controller/Pawn'da ekle.
- BindAction ile Input Action'ı bir C++ fonksiyonuna bağla.

⚠ Kısıtlar

- Mapping Context'i eklemeyi unutma — Input Action tek başına hiçbir şey tetiklemez.

✓ Beklenen Sonuç

- E tuşuna basıldığında bağlı C++ fonksiyonu çalışıyor (bir UE_LOG ile doğrula).

EXERCISE 05 · 20 XP

Etkileşim İşını

İlgili dersler: Line and Shape Tracing · Sphere Collision Shape · Casting

Amaç — Oyuncunun baktığı yöne bir küre-şekilli trace göndererek önündeki toplanabilir eşyayı tespit etmek.

► Yapılacaklar

- Kamera konumu ve bakış yönünden başlangıç/bitiş noktalarını hesapla.
- SweepSingleByChannel'ı bir FCollisionShape::MakeSphere ile çağır.
- Sonucu bir FHitResult referansına yaz, vurulan Actor'ü Cast<> ile eşya tipine dönüştürmeyi dene.

⚠ Kısıtlar

- Cast<> sonucunu if ile kontrol etmeden kullanma.

✓ Beklenen Sonuç

- Sadece gerçek bir toplanabilir eşyaya bakıldığında etkileşim tetikleniyor, boşluğa bakıldığında değil.

EXERCISE 06 · 20 XP

Envanter Sistemi

İlgili dersler: TArray · Removing From Array · Taking Items Back

Amaç — Toplanan eşyaları bir TArray'de tutan, kilide yerleştirilince çıkaran basit bir envanter yazmak.

► Yapılacaklar

- Oyuncu Controller/Pawn'da bir TArray envanter değişkeni tanımla.
- Eşya toplanınca diziye ekle, dünyadan gizle/yok et.
- Kilide yerleştirilince diziden Remove et.

⚠ Kısıtlar

- Aynı eşyayı diziye birden fazla kez eklemeyi engellemeyi unutma.

✓ Beklenen Sonuç

- Envanter içeriği (ör. UE_LOG ile) her toplama/bırakma sonrası doğru güncelleniyor.

EXERCISE 07 · 25 XP

Kilit ve Gizli Duvar

İlgili dersler: Lock Blueprint · Secret Wall · Escaping The Dungeon

Amaç — Şimdiye kadarki tüm sistemleri birleştirip gizli bir duvarın açılmasını ve zindandan kaçışı kurmak.

► Yapılacaklar

- Kilit Actor'üne, doğru eşya yerleştirilince açılan bir mantık ekle (delegate ile).
- Kilit açıldığında gizli duvarın görünürlüğü/collision'ını değiştir.
- Baştan sona bir oynanış testi yap: plaka → kapı → eşya → kilit → gizli duvar → çıkış.

⚠ Kısıtlar

- Entegrasyonu parça parça değil, tek seferde “umarım çalışır” diye test etme.

✓ Beklenen Sonuç

- Oyuncu, tüm zinciri sırasıyla tamamlayıp zindandan çıkabiliyor.

BÖLÜM II — BONUS (İSTEĞE BAĞLI)

Bonus, zorunlu kısım tam ve eksiksiz çalışır durumda olmadan başlanmamalıdır — tıpkı 42'deki mantık gibi: önce sağlam temel, sonra süsleme.

BONUS · +35 XP (zorunlu %100 tamamlanınca açılır)

Çift Plaka Kapısı

Bağımsız pratik — delegate ve sayaç mantığını genişletme

Amaç — Sadece iki basınç plakası aynı anda basılıyken açılan, kooperatif bir kapı mekaniği kurmak.

► Yapılacaklar

- İki ayrı basınç plakası yerleştir, her biri kendi delegate'ini yayınlasın.
- Kapı Actor'ü her iki plakanın da aktif olduğu durumu takip eden bir mantık tutsun.
- Sadece ikisi de aktifken kapıyı aç.

⚠ Kısıtlar

- Kapının açılma kararını tek bir plakanın delegate'ine bağlama — her iki durumu da izlemeli.

✓ Beklenen Sonuç

- Tek plaka basılıyken kapı kapalı kalıyor, ikisi birden basılıyken açılıyor.

TESLİM VE ÖZ-DEĞERLENDİRME

Bu bölüm bittiğinde composition, pointer ve delegate artık soyut kavramlar değil, kendi yazdığın çalışan bir sistemin parçaları olmalı. Bir sonraki bölümde (Battle Blaster) aynı composition mantığını hasar sistemi için kullanacaksın — burada Health/Trigger ayrımını iyi anladıysan orası çok daha kolay gelecek.